

Avaliação Técnica: Desenvolvedor(a) Fullstack Sênior

Objetivo

Avaliar o raciocínio técnico, qualidade de implementação, maturidade arquitetural, clareza de comunicação e o pragmatismo na entrega de uma solução ponta a ponta.

Formato e Prazos

1. **Perguntas abertas:** Raciocínio técnico e experiência prática.
2. **Desafio Prático:** Implementação de um dos cenários propostos (Back-end, Front-end e Infra).
3. **Apresentação:** Demonstração da solução e defesa das decisões técnicas em entrevista.
 - **Tempo estimado:** 4 a 6 horas.
 - **Prazo sugerido:** Até 4 dias corridos.
 - **Uso de IA:** O uso de IA generativa e consultas externas é permitido. O foco da avaliação é o seu **critério técnico** e a capacidade de validar a qualidade e segurança do que foi gerado.

1. Raciocínio Técnico (Perguntas Abertas)

Responda de forma objetiva no seu README final (máximo de 10 linhas por resposta):

1. **Integração Resiliente:** Como você desenharia uma integração com uma API externa que possui limites de requisição (*rate limiting*) e instabilidade ocasional para garantir que seu sistema continue funcional?
2. **Refinamento de Requisito:** Ao receber uma demanda vaga da área de negócio, quais etapas você segue para transformá-la em uma especificação técnica pronta para desenvolvimento?
3. **Idempotência:** Em uma API de pagamentos ou pedidos, como você evita que o processamento seja duplicado em caso de retentativas do cliente?
4. **Síncrono vs. Assíncrono:** Quais critérios definem se um fluxo deve ser resolvido imediatamente na requisição HTTP ou processado em segundo plano (fila)?
5. **Segurança:** Quais controles mínimos de segurança você aplica em uma API exposta publicamente?
6. **Qualidade e Entrega:** Como você decide o que é essencial para uma primeira versão (MVP) e o que deve ser tratado como débito técnico ou melhoria futura?
7. **Governança e IA:** Como utilizar IA para acelerar o desenvolvimento sem comprometer a segurança dos dados e a qualidade do código a longo prazo?

2. Desafio Prático

Instrução Principal

Abaixo, apresentamos 3 cenários de negócio. **Você deve escolher APENAS UM deles para implementar.** O objetivo é construir uma plataforma integrada (Front + Back + Banco) que envolva persistência local e integração com uma API pública.

Escolha um cenário:

Cenário	O que construir	O que será avaliado
1. Gestão de Tickets	Módulo para cadastro, listagem, atualização de status e prioridade de tickets de suporte.	Regras de status, noção de SLA/prioridade e histórico básico.
2. Mini CRM	Cadastro e manutenção de usuários/clientes com perfis, contatos e status de relacionamento.	Estruturação de entidades, consistência de validações e visão de negócio.
3. Catálogo de Itens	Cadastro, edição e consulta de produtos com categorização e busca avançada.	Modelagem flexível, filtros de pesquisa e padronização de dados (SKU/Preço).

Requisitos Técnicos (Obrigatórios)

Para o cenário escolhido, sua solução deve contemplar:

Back-End (Node.js)

- **Arquitetura:** Separação entre a API (HTTP) e o Worker de processamento.
- **Processamento Assíncrono:** Ao criar um registro, o "processamento pesado" ou a integração com a API externa escolhida deve ocorrer via **Fila** (ex: BullMQ, RabbitMQ, Redis).
- **Resiliência:** Implementar lógica de **Retry** automático em caso de falha no worker.
- **API externa:** Integrar com qualquer API pública para enriquecer os dados do seu sistema.

Front-End (React)

- **Interface:** SPA simples para cadastro e visualização.
- **UX Assíncrona:** Como o processamento ocorre em fila, a lista deve refletir a mudança de status (ex: "Pendente" -> "Processado") sem que o usuário precise dar F5 na página.

- **Filtros:** Implementar busca e filtros (por status, categoria ou nome).

Infraestrutura (Docker)

- **Execução:** O projeto deve subir inteiramente com um único comando: `docker-compose up`.
- **Serviços:** O compose deve orquestrar: App, Front, Banco de Dados e o serviço de Mensageria (Fila).



Critérios de Avaliação

- **Maturidade Sênior:** Organização de pastas, Clean Code e uso de Design Patterns.
- **Tratamento de Erros:** Como o sistema lida com falhas na API externa ou entradas inválidas.
- **Performance:** Eficiência na listagem de dados (paginação) e uso de variáveis de ambiente.
- **Testes:** Presença de testes (unitários ou integração) para fluxos críticos.



Entregáveis

1. **Código-fonte:** Repositório no GitHub ou GitLab (público ou com acesso liberado).
2. **README:**
 - a. Instruções claras de execução via Docker.
 - b. Respostas das perguntas da **Parte 1**.
 - c. Breve explicação das decisões técnicas e trade-offs realizados.
 - d. O que você faria de diferente ou melhoria se tivesse mais tempo ou se o sistema tivesse 1 milhão de acessos.
3. **Demonstração:** Um vídeo curto (opcional) ou apresentação da solução durante a entrevista.

Boa sorte! Estamos ansiosos para ver sua solução.